Introduction

Crime remains a pressing issue in South Africa, impacting not only the safety and well-being of communities but also hindering economic growth and national development. To address this, policymakers and law enforcement agencies increasingly rely on data-driven insights to inform their decisions. This project reflects my interest in combining geospatial analysis and time series forecasting to support public safety efforts. I aim to explore the development of a predictive model that captures both spatial and temporal crime patterns. By doing so, there is potential to uncover meaningful trends, anticipate future crime hotspots.

Objectives:

- Preprocess and analyze historical crime data.
- Forecast future crime levels using time series modeling.
- Map forecast results using geospatial techniques.
- Visualize trends to support decision-making.

Background Theory

<u>Time Series Forecasting</u>

According to Hyndman and Athanasopoulos (2018), time series forecasting is a method used to predict future values based on previously observed data points collected over time. In this project, I applied time series forecasting to crime statistics, which are naturally recorded over sequential time periods such as each year. This approach not only helped me understand how crime levels have changed historically, but also provided insight into where they might be heading in the future.

To implement the forecasting, I used Facebook Prophet — a powerful and flexible time series model developed by Meta (Meta, 2017). Prophet breaks down time series data into trend (long-term direction), seasonality (recurring patterns), and holidays or special events (unexpected disruptions). I found Prophet particularly effective for handling challenges such as missing values and irregular time intervals, which are common in real-world crime datasets.

By using this model, I was able to forecast future crime rates at the level of individual police stations. This is useful for moving toward predictive policing, where data is used to inform planning and help law enforcement allocate resources more efficiently, potentially improving public safety outcomes (Hyndman and Athanasopoulos, 2018; Meta, 2017).

Geospatial Data Analysis

Geospatial data analysis is the study of data linked to specific locations on the Earth's surface. It involves examining and interpreting information associated with geographic coordinates, such as latitude and longitude. According to Longley et al. (2015), this type of analysis allows for a deeper understanding of spatial relationships and patterns that would be difficult to detect in non-spatial datasets. In the context of crime statistics, geospatial analysis enables us to understand where crimes occur, how they are distributed across regions, and which areas are most affected.

In this project, I applied geospatial analysis to crime data from South Africa by mapping incidents to their corresponding police stations. This was essential because crime does not occur randomly — it often clusters in particular areas due to social, economic, and environmental influences (Chainey & Ratcliffe, 2013). For instance, urban centres may exhibit higher crime rates than rural areas, and certain provinces may be more prone to specific types of crime.

Using tools such as Folium and Marker Clusters, I developed interactive maps to visualize spatial crime patterns. These maps allowed me to highlight stations with the highest crime forecasts and visually compare predicted versus actual crime counts. Although I did not use a heatmap in this project, color-coded and clustered markers served a similar purpose by indicating the intensity and distribution of crime geographically.

Geospatial analysis also improves predictive modeling by incorporating location-based features such as latitude and longitude, into machine learning models. This integration helps forecast not only when crime might occur but also where, which is particularly useful for targeted policing strategies and resource deployment (Malleson & Andresen, 2016).

In this project, crime counts are tied to police station coordinates, allowing us to:
- Visualize spatial patterns of crime.
- Identify Top 3 crime areas.
- Map forecasted crime trends geographically.

Implementation

Dataset

Sources:
- Crime data from Kaggle (South African police crime statistics)
- Police station coordinates from Open.Africa:

File: crime_stats_with_locations.csv

Features:
- Station name
- Province

- Latitude and Longitude
- Crime Category
- Yearly crime counts (e.g., 2017-2018)

Preprocessing Steps

- Removed unnamed or irrelevant columns.
- Melted wide-format year columns into long-format time series.
- Converted year strings to midpoints (e.g., '2017-2018' → 2017.5).
- Grouped by station and category to prepare input for forecasting.

Forecasting with Prophet

- Time series models were trained per station using Prophet.
- Each model forecasted 5 additional years (to ~2026.5).
- Forecast outputs included trend and uncertainty intervals.
- Results saved in: station_crime_forecasts_corrected.csv

Geospatial Mapping

- Combined forecasts with geographic coordinates.
- Exported merged data as crime_forecast_map_data.csv.
- Prepared for mapping using Folium or GIS tools.

Tools and Libraries

Library	**Purpose**
pandas	Data manipulation and cleaning
prophet	Time series forecasting
matplotlib	Trend visualization
folium	Geospatial interactive maps
datetime	Handling time data

Results

National Crime Trends

-A line chart of total national crime counts (2017.5 to 2021.5) revealed:
- Slight downward trend across the years.
- Notable annual variability, likely tied to socio-political and economic events.

Station-Level Forecasts

- Forecasts predict crime trends for individual police stations.
- Stations in urban provinces (e.g., Gauteng) show higher and more volatile crime trends.
- Rural stations show more stable but sometimes rising forecasts.

Geospatial Insights

- Geospatial plots highlight crime hotspots.
- Merged forecast results allow visual exploration of where crime may rise.
- Supports resource allocation for law enforcement.

Limitations

- Some crime records were missing or incomplete.
- Forecasts do not yet incorporate external factors (e.g., unemployment, urban growth).
- The year labels ('2017-2018') are approximated as 2017.5, which may introduce slight time alignment errors.

Code Documentation

I used google colab for this project and it contains different sections.

Section 1: Population Map

This code creates an interactive population map of South African provinces using Folium. Each province is represented by a blue circle, and the size of the circle reflects the population of the province.

Having a population map is essential when analyzing and forecasting crime because it provides critical context that raw crime numbers alone can't offer. For example, if a province like Gauteng shows a high number of crimes, that might seem alarming—but it's also one of the most densely

populated areas in South Africa. Without accounting for population size, we risk misinterpreting the data.

```
#population map creation

import pandas as pd
import folium

# Load data
df = pd.read_excel("ProvincePopulation (2).xlsx")
df.columns = df.columns.str.strip().str.lower()

# folium map
m = folium.Map(location=[-30.5595, 22.9375], zoom_start=6)

# Add population circles
for _, row in df.iterrows():
    folium.CircleMarker(
        location=[row['latitude'], row['longitude']],
        radius=row['population'] / 1_000_000,
        popup=f"{row['province']}: {row['population']:,}",
        color='blue',
        fill=True,
        fill_opacity=0.6
    ).add_to(m)

# Save and open map
m.save("province_population_map.html")
```

Section 2:Loading files and checking index

At this point in the project, I loaded the crime statistics Excel file using pandas, and then I printed out the column names. This might seem like a small step, but it's actually really important. It helps me understand what kind of data I'm working with—like which columns represent crime categories, which ones have the years, and whether there's any identifying info like police station names or provinces. It also helps me catch any issues early, like misspelled column names, unexpected formatting, or extra columns I don't need. By seeing the column names upfront, I can plan how to clean the data and make sure I'm referencing everything

correctly in the rest of the code. It's a quick check, but it saves a lot of time and prevents errors later on.

```python
import pandas as pd

# Load both CSV files
crime_stats = pd.read_excel('SouthAfricaCrimeStats.xlsx')
stations_locations = pd.read_csv('geo_export_b681af05_ce59_42e8_87e6_41af282b0a6e.csv')


crime_stats = pd.read_excel('SouthAfricaCrimeStats.xlsx')
print(crime_stats.columns)
```

```
Index(['Province', 'Station', 'Longitude', 'Unnamed: 3', 'Category',
       '2005-2006', '2006-2007', '2007-2008', '2008-2009', '2009-2010',
       '2010-2011', '2011-2012', '2012-2013', '2013-2014', '2014-2015',
       '2015-2016'],
      dtype='object')
```

Section 3: Merging Data

To begin the analysis, I loaded two key datasets: one containing South African crime statistics and another with police station geographic coordinates. This was the foundation for combining crime data with geospatial information.

```python
# Merge on cleaned station names
merged = pd.merge(
    crime_stats,
    stations_locations[['station_clean', 'Latitude', 'Longitude']],
    on='station_clean',
    how='left',
    suffixes=('', '_from_locations')
)

# Handle longitude (has suffix due to conflict)
merged['Longitude'] = merged['Longitude'].combine_first(merged['Longitude_from_locations'])

# Latitude was added directly (no suffix), so no need to rename
# Drop temp columns
merged.drop(columns=['station_clean', 'Longitude_from_locations'], inplace=True)

# Save to new CSV
merged.to_csv('crime_stats_with_locations.csv', index=False)
print("✅ Successfully saved as 'crime_stats_with_locations.csv'")
```

```
✅ Successfully saved as 'crime_stats_with_locations.csv'
```

Section 4: Predictive model

In this part of the project, I began by importing essential Python libraries such as pandas and numpy for data handling, and several scikit-learn modules for building and evaluating the machine learning model. I then loaded the dataset containing South African crime statistics with geographic coordinates. To clean the data, I removed an unnecessary column that appeared due to formatting issues in the original CSV. After that, I reshaped the dataset from wide format to long format using the melt function. This transformation was necessary because the original data had separate columns for each year, and converting it into a long format—with one row per station, category, and year—makes it much easier to use for time series forecasting and regression modeling. This step was foundational for preparing the data to train a predictive model that can analyze crime trends over time and across locations.

```
[26] import pandas as pd
     import numpy as np
     from sklearn.model_selection import train_test_split
     from sklearn.ensemble import GradientBoostingRegressor
     from sklearn.preprocessing import LabelEncoder
     from sklearn.metrics import mean_squared_error, r2_score

     # Load dataset
     df = pd.read_csv("crime_stats_with_locations.csv")

     # Drop unnecessary column
     df = df.drop(columns=["Unnamed: 3"])

     # Melt into long format
     df_melted = df.melt(
         id_vars=['Province', 'station', 'Longitude', 'Latitude', 'Category'],
         var_name='Year'
```

In this stage of the project, I began by cleaning the reshaped dataset. I removed any rows containing missing values using dropna() to ensure the model only trains on complete data. Then, I converted the year column, which was originally in the format like "2017-2018", into a single numeric value by averaging the two years. This step helps the model interpret time as a continuous numerical feature. After that, I selected relevant features—such as province, police station, crime category, geographic coordinates, and year—for modeling, with the crime count as the target variable. Since machine learning models cannot handle categorical text data directly, I used LabelEncoder to convert the province, station, and category columns into numeric labels. Once the data was prepared, I split it into training and testing sets using an 80/20 ratio to evaluate model performance reliably. I trained a Gradient Boosting Regressor, which is a powerful ensemble learning method that builds multiple decision trees to improve accuracy. Finally, I used the test data to make predictions and evaluated the model using RMSE (Root Mean Squared Error) and R^2 score. These metrics helped measure how well the model captured the patterns in the crime data, with the R^2 score in particular showing the proportion of variance in the crime counts that the model could explain.

```python
df_melted.dropna(inplace=True)
df_melted['Year'] = df_melted['Year'].apply(lambda x: sum(map(int, x.split('-'))) / 2)

# Features and target
X = df_melted[['Province', 'station', 'Category', 'Latitude', 'Longitude', 'Year']]
y = df_melted['Crime_Count']

# Label encode categorical features
categorical_features = ['Province', 'station', 'Category']
label_encoders = {}
for col in categorical_features:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    label_encoders[col] = le

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = GradientBoostingRegressor(n_estimators=50, random_state=42)
model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"RMSE: {rmse:.2f}")
print(f"R² Score: {r2:.2f}")
```

Section 5: Visualize and predict

In this part of the project, I focused on preparing the data for model training and improving the performance of the crime prediction model. First, I dropped all rows with missing values using dropna() to avoid feeding incomplete or inaccurate data into the model, which could lead to unreliable predictions. Next, I converted the year column from a string format like "2017-2018" into a numeric value by averaging the two years (e.g., 2017.5). This is important because machine learning models require numerical input for continuous features like time.

To speed up processing and improve visualization performance, especially during development, I sampled 1,000 rows randomly from the dataset. This still provided a diverse and representative subset of the data while reducing computation time. I then selected relevant features such as province, station, crime category, geographic location (latitude and longitude), and year, with the crime count set as the target variable to predict. Since categorical features like province or station are in text format, I used LabelEncoder to convert them into numeric values so the model could interpret them properly.

I split the dataset into training and testing sets, using 80% of the data for training and 20% for testing. This ensures the model is evaluated on data it hasn't seen before, providing a realistic measure of its accuracy. Finally, I trained a Gradient Boosting Regressor—a robust ensemble model known for its predictive power—and used it to forecast crime counts. I evaluated its

performance using RMSE (which shows the typical error in predictions) and the R^2 score (which measures how well the model explains the variation in the data). These steps were crucial for building a solid predictive model that could generalize well to new, unseen crime data.

```
# Drop missing values and convert years
df_melted.dropna(inplace=True)
df_melted['Year'] = df_melted['Year'].apply(lambda x: sum(map(int, x.split('-'))) / 2)

# Sample 1000 rows for faster visualization
df_sampled = df_melted.sample(n=1000, random_state=42)

# Feature Engineering
X = df_sampled[['Province', 'station', 'Category', 'Latitude', 'Longitude', 'Year']]
y = df_sampled['Crime_Count']

# Encode categorical features
categorical_features = ['Province', 'station', 'Category']
label_encoders = {}
for col in categorical_features:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    label_encoders[col] = le

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Model
model = GradientBoostingRegressor(n_estimators=50, random_state=42)
model.fit(X_train, y_train)

# Predict and Evaluate
y_pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
```

The next part of the code evaluates the machine learning model's performance and visualizes its predictions on an interactive map. It starts by printing performance metrics like RMSE and R^2 Score to assess how well the model predicted crime counts. Then, it prepares the test results by adding predicted and actual values and converting station IDs back to their names. The continued code is in the python file.

An interactive Folium map was created, where each police station is marked. The markers are color-coded based on prediction accuracy—green for accurate, orange for moderate error, and red for high error—providing a visual indicator of model reliability by location. Finally, the map is saved as an HTML file, making it easy to share and explore. This step combines machine learning and geospatial analysis to help visualize where the model performs well and where improvements might be needed.

Section 6: Forecast

At this stage, what I did was prepare the data for time series forecasting using a tool called Facebook Prophet. First, I took the year ranges in the dataset — like "2017 to 2018" — and converted those into midpoints, such as 2017.5, to represent a single point in time. Then I went a step further and transformed those into actual date formats, because Prophet needs proper timestamps to model the trends accurately.

After that, I grouped the data by police station and year so we could see the total number of crimes reported per year at each station. Once the data was structured properly, I selected the top three stations that had the most complete time series data — just to make sure we were working with stations that had enough history to make meaningful forecasts.

Finally, I trained a Prophet model for each of those top stations. This allows us to look ahead and predict how crime might increase or decrease over the next few years in specific areas — which is really useful for planning, allocating resources, or even just understanding the broader trends.

```
# Convert year range like "2017-2018" into midpoint float like 2017.5
df_long['Year'] = df_long['Year'].apply(lambda x: sum(map(int, x.split('-'))) / 2 if '-' in x else float(x))

# Convert float year to datetime
def year_fraction_to_date(year_frac):
    year = int(year_frac)
    fraction = year_frac - year
    days_in_year = 366 if pd.Timestamp(f"{year}-12-31").is_leap_year else 365
    day_of_year = int(fraction * days_in_year)
    return pd.Timestamp(f"{year}-01-01") + pd.to_timedelta(day_of_year, unit='D')

# Group by station and year
grouped = df_long.groupby(['station', 'Year'])['Crime_Count'].sum().reset_index()
grouped.columns = ['station', 'ds', 'y']
grouped['ds'] = grouped['ds'].apply(year_fraction_to_date)

# Select top 3 stations
top_stations = grouped['station'].value_counts().head(3).index.tolist()

results = []

for station in top_stations:
    station_df = grouped[grouped['station'] == station].copy()

    if station_df['ds'].nunique() < 2:
        continue

    model = Prophet(yearly_seasonality=True)
    model.fit(station_df)
```

Please refer to the python file for full code

Section 7:National trend over time analysis

In this part of the project, I wanted to visualize the overall national crime trend over time across all police stations. First, I loaded the dataset containing crime statistics with location details. I then cleaned the dataset by removing any unnecessary columns — especially those that were automatically generated like "Unnamed: 0".

Next, I identified all the columns that represent crime years, which are labeled in a range format like "2017-2018". I converted the dataset from a wide format — where each year is a separate column — into a long format, where each row represents a single crime count for a specific station, category, and year.

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
file_path = "crime_stats_with_locations.csv"  # Update if needed
df = pd.read_csv(file_path)

# Remove unnecessary unnamed columns (like "Unnamed: 0")
df = df.loc[:, ~df.columns.str.contains("^Unnamed")]

# Identify year columns (e.g., '2017-2018', etc.)
year_columns = [col for col in df.columns if '-' in col]

# Convert from wide to long format
df_long = df.melt(
    id_vars=['Province', 'station', 'Latitude', 'Longitude', 'Category'],
    value_vars=year_columns,
    var_name='Year',
    value_name='Crime_Count'
)

# Convert year ranges to midpoints (e.g., 2017.5)
df_long['Year'] = df_long['Year'].str.extract(r'(\d{4})').astype(float) + 0.5

# Ensure crime count is numeric
df_long['Crime_Count'] = pd.to_numeric(df_long['Crime_Count'], errors='coerce').fillna(0)

# Group and sum by year to get national totals
national_trend = df_long.groupby('Year')['Crime_Count'].sum().reset_index()

# Plot the national trend
plt.figure(figsize=(10, 6))
plt.plot(national_trend['Year'], national_trend['Crime_Count'], marker='o', linestyle='-', color='crimson')
plt.title('National Crime Trend Over Time (All Stations)')
plt.xlabel('Year')
plt.ylabel('Total Crime Count')
plt.grid(True)
plt.tight_layout()
plt.show()
```

To make the year values numeric and suitable for plotting, I converted those year ranges into midpoints — for example, "2017-2018" became 2017.5. I also made sure all crime counts were treated as numeric values and filled in any missing or non-numeric entries with zero to avoid issues during aggregation.

After that, I grouped the data by year and summed up all crime counts nationally, so we get a single total for each year. Finally, I plotted the national crime trend using Matplotlib. The result is a clear line chart showing how total crime has increased or decreased over the years, which is helpful for spotting any national patterns or shifts in crime over time.

Conclusion

This project successfully combined time series forecasting with geospatial data to predict crime patterns in South Africa. It provides:

- Accurate, interpretable forecasts for future crime levels.
- Geographic mapping of trends to support proactive intervention.
- A foundation for further modeling using socioeconomic and demographic features.

References

Anselin, L. (1995) 'Local Indicators of Spatial Association (LISA)', *Geographical Analysis*, 27(2), pp. 93–115.

Chainey, S. and Ratcliffe, J. (2013) *GIS and Crime Mapping*. 2nd edn. London: Wiley.

Facebook Prophet (n.d.) *Prophet: Forecasting at scale*. Available at: https://facebook.github.io/prophet/ (Accessed: 6 June 2025).

Kaggle (n.d.) *South African Crime Statistics*. Available at: https://www.kaggle.com/datasets (Accessed: 11 June 2025).

Open.Africa (n.d.) *Police Station Coordinates Dataset*. Available at: https://open.africa/dataset/police-station-coordinates/resource/a1c2c4d9-04a5-46ae-be2e-50984293db7d (Accessed: 11 June 2025).

South African Police Service (n.d.) *Crime Statistics*. Available at: https://www.saps.gov.za/services/crimestats.php (Accessed: 6 June 2025).